

Práctica 1: Dockerización del despliegue de una aplicación Node.js

Introducción

En este caso vamos a Dockerizar la aplicación que ya desplegamos en la práctica de node.js.

¿Por qué *dockerizar*?

[Si uno trata de informarse](#), encontrará [múltiples y variadas razones](#) para dockerizar nuestras aplicaciones y servicios.

Por citar sólo algunas:

- 1. Configuración rápida del entorno en local para el equipo de desarrollo:** si todos los servicios están implementados con contenedores, es muy rápida la configuración de dicho entorno.
- 2. Evita el clásico "en mi máquina funciona":** gran parte de los problemas de desarrollo provienen de la propia configuración que los integrantes del equipo de desarrollo tienen de su entorno. Con los servicios en contenedores, esto queda solucionado en gran medida.
- 3. Despliegues más rápidos**
- 4. Mejor control de versiones:** como ya sabéis, se puede etiquetar (tags), lo que ayuda en el CONTROL DE VERSIONES.
- 5. Rollbacks más fáciles:** puesto que se tienen las cosas más controladas por la versión, es más fácil revertir el código. A veces, simplemente apuntando a su versión de trabajo anterior.
- 6. Fácil configuración de múltiples entornos:** como hacen la mayoría de los equipos de desarrollo, se establece un entorno local, de integración, de puesta en escena (preprod) y de producción. Esto se hace más fácil cuando los servicios están en contenedores y, la mayoría de las veces, con sólo un cambio de VARIABLES DE ENTORNO.
- 7. Apoyo de la comunidad:** existe una fuerte comunidad de ingenieros de software que continuamente contribuyen con grandes imágenes que pueden ser reutilizadas para desarrollar un gran software. ¿Por qué reinventar la rueda, no?

Despliegue con Docker

En primer lugar, si eliminastéis el repositorio en su momento, debéis volver a clonarlo en vuestra Debian, en caso contrario obviad este paso:

```
git clone https://github.com/contentful/the-example-app.nodejs.git
```

Ahora, puesto que la aplicación ya viene con el Dockerfile necesario dentro del directorio para construir la imagen y correr el contenedor, vamos a estudiar su contenido.

Tarea

Completa este Dockerfile con las opciones/directivas adecuadas, leed los comentarios y podéis apoyaros en la teoría, en [este cheatsheet](#), en [este otro](#) o en cualquiera que encontréis.

```
_____ node:9 #  
_____ /app #  
_____ npm install -g contentful-cli #  
_____ package.json . #  
_____ npm install #  
_____ . . #  
_____ node #  
_____ 3000 #  
_____ ["npm", "run", "start:dev"] #
```

Nota

En Linux, cuando queremos hacer referencia al directorio actual, lo hacemos con un punto.

Si dentro de nuestro directorio actual tenemos una carpeta llamada prueba, podemos hacer referencia a ella como ./prueba, ya que el . hace referencia precisamente al directorio donde nos encontramos

Así pues, tener nuestra aplicación corriendo es cuestión de un par de comandos.

Hacemos un build de la imagen de Docker. Le indicamos que ésta se llama the-example-app.nodejs y que haga el build con el contexto del directorio actual de trabajo, así como del Dockerfile que hay en él:

```
docker build -t the-example-app.nodejs .
```

Y por último, iniciamos el contenedor con nuestra aplicación. Ahora sí, con la opción -p, le indicamos que escuche conexiones entrantes de cualquier máquina en el puerto 3000 de nuestra máquina anfitrión que haremos coincidir

con el puerto 3000 del contenedor (-p 3000:3000). Y con la opción -d lo haremos correr en modo demonio, en background:

```
docker run -p 3000:3000 -d the-example-app.nodejs
```

Tras esto sólo queda comprobar que, efectivamente, desde nuestra máquina podemos acceder a: http://IP_Maq_Virtual:3000 y que allí está nuestra aplicación en funcionamiento.

Tarea

Documenta, incluyendo capturas de pantallas, el proceso que has seguido para realizar el despliegue de esta nueva aplicación, así como el resultado final.

Práctica 2: Despliegue de una aplicación PHP con Nginx y MySQL usando Docker y docker-compose

Introducción

¡Atención!

En caso de que tengáis problemas, esta práctica está comprobada y funcionando usando las siguientes versiones:

- **Docker:** Docker version 20.10.17, build 100c701
- **Docker-compose:** Docker Compose version v2.10.2

Recordando qué es *docker-compose*

Para ejecutar nuestra aplicación en docker creamos un fichero llamado `Dockerfile` y este fichero contiene una configuración. Esta configuración varía dependiendo de qué queremos poner en el contenedor, ya que no es lo mismo poner una página web, que una base de datos.

Este proceso, de crear todos los `Dockerfile` y ejecutarlos puede ser bastante tedioso, ya que debemos pensar que una aplicación de tamaño mediano es probable que tenga un front end, un back end, quizá algunos background-workers así como la base de datos, sistema de caché, sistema de colas o de message-broker... por lo que cada uno de nuestros servicios será un contenedor diferente.

Por lo tanto, crear múltiples `Dockerfile` y ejecutarlos todo en un script queda largo y feo.

Aquí es donde entra `docker-compose` el cual es una herramienta que nos permite definir y correr múltiples contenedores en Docker. Estos múltiples contenedores se definen en un fichero denominado `docker-compose` con la extensión `.yaml`. Luego, con un solo comando, crea e inicia todos los servicios desde su configuración.



Compose funciona en todos los entornos: producción, puesta en escena, desarrollo, pruebas, así como flujos de trabajo de CI.

Usar Compose es básicamente un proceso de tres pasos:

- Definir el entorno de nuestra aplicación con un Dockerfile para que pueda reproducirse en cualquier lugar.
- Definir los servicios que componen la aplicación `docker-compose.yml` para que puedan ejecutarse juntos en un entorno aislado.
- Ejecutar `docker-compose up` y Compose inicia y ejecuta toda su aplicación.

Este proceso se denomina orquestación de contenedores y se lleva a cabo de forma local al interior de los containers, quienes, además, se encontrarán unidos a través de una red de Docker.

Instalación de *docker-compose*

Proceso de dockerización de Nginx+PHP+MySQL

1. Estructura de directorios

Para que quede claro todo el proceso que vamos a seguir, la estructura de directorios que nos debe quedar en nuestra Debian al finalizar la práctica es esta:

```
/usuario/home/practica6-2/
├── docker-compose.yml
├── nginx
│   ├── default.conf
│   └── Dockerfile
├── php
│   └── Dockerfile
├── www
│   ├── html
│   └── index.php
```

Podéis ir creando los directorios y archivos paso a paso o crearlo todo a la vez y luego ir rellenando los archivos vacíos siguiendo un procedimiento como este:

```
mkdir practica6-2
cd practica6-2
touch docker-compose.yml
mkdir nginx
touch nginx/default.conf
...
```

2. Creación de un contenedor Nginx

Para empezar, necesitamos crear y correr un contendor Nginx que permita alojar nuestra aplicación en PHP.

Dentro de la carpeta `/usuario/home/practica6-2/` debemos haber creado o crear ahora el archivo `docker-compose.yml`

Y editamos este archivo con el editor de texto que prefiramos, nano por ejemplo:

```
nano docker-compose.yml
```

Y añadimos las siguientes líneas:

```
nginx:
  image: nginx:latest
  container_name: nginx-container
  ports:
    - 80:80
```

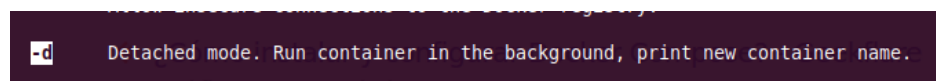
Y lo guardamos.

El archivo que acabamos de crear será el encargado de descargarse la última versión de la imagen de Nginx, crear un contenedor con ella y publicar o escuchar en el puerto 80 del contenedor que también se corresponderá con el 80 de nuestra máquina (80:80).

Iniciemos entonces este proceso:

```
docker-compose up -d
```

Con la opción `-d` (de daemon), estamos indicando que el contenedor se ejecute en background o segundo plano:



Para comprobar que el contenedor está corriendo, podemos hacer:

```
docker ps
```

Y deberíamos ver algo como:

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
NAMES					
c6641e4d5bbf	nginx:latest	"/docker-entrypoint..."	5 seconds ago	Up 3 seconds	0.0.0.0:80->80/tcp, :::80->80/tcp
nginx-container					

Además, si abrimos el navegador de nuestra máquina anfitrión y accedemos a `http://IP_Maq_Virtual` deberíamos ver la página de bienvenida de Nginx:

Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

3. Creación de un contenedor PHP

Creamos la carpeta y el documento pertinente dentro de ella, si no lo habíamos hecho antes:

```
mkdir -p /home/usuario/practica6-2/www/html
nano /home/usuario/practica6-2/www/html/index.php
```

Y dentro de `index.php` añadimos el siguiente código:

```
<!DOCTYPE html>
<head>
  <title>¡Hola mundo!</title>
</head>

<body>
  <h1>¡Hola mundo!</h1>
  <p><?php echo 'Estamos corriendo PHP, version: ' . phpversion(); ?></p>
</body>
```

Guardad el archivo y cread, si no lo habíais hecho antes, un directorio llamado *nginx* dentro del directorio del proyecto:

```
mkdir /home/usuario/practica6-2/nginx
```

Ahora vamos a crear el archivo de configuración por defecto para que Nginx pueda correr la aplicación PHP:

```
nano /home/usuario/practica6-2/nginx/default.conf
```

Y dentro de ese archivo, colocaremos la siguiente configuración:

```
server {

    listen 80 default_server;
    root /var/www/html;
    index index.html index.php;

    charset utf-8;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location = /favicon.ico { access_log off; log_not_found off; }
    location = /robots.txt { access_log off; log_not_found off; }

    access_log off;
    error_log /var/log/nginx/error.log error;

    sendfile off;

    client_max_body_size 100m;

    location ~ .php$ {
        fastcgi_split_path_info ^(.+\.php)(/.+)$;
        fastcgi_pass php:9000;
        fastcgi_index index.php;
        include fastcgi_params;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
        fastcgi_intercept_errors off;
        fastcgi_buffer_size 16k;
        fastcgi_buffers 4 16k;
    }
}
```

```
location ~ /.ht {
    deny all;
}
```

Guardamos el archivo y ahora crearemos el `Dockerfile` dentro del directorio *nginx*. En este archivo se copiará el archivo de configuración de Nginx al contenedor correspondiente.

Así pues:

```
nano /home/usuario/practica6-2/nginx/Dockerfile
```

Y dentro de este archivo:

```
FROM nginx:latest
```

```
COPY ./default.conf /etc/nginx/conf.d/default.conf
```

Y ahora editamos nuestro archivo `docker-compose.yml`:

```
services:
  nginx:
    build: ./nginx/
    container_name: nginx-container
    ports:
      - 80:80
    links:
      - php
    volumes:
      - ./www/html:/var/www/html/

  php:
    image: php:7.0-fpm
    container_name: php-container
    expose:
      - 9000
    volumes:
      - ./www/html:/var/www/html/
```

Ahora con este fichero `docker-compose.yml` se creará un nuevo contenedor PHP-FPM en el puerto 9000, enlazará el contenedor *nginx* con el contenedor *php*, así como creará un volumen y lo montará en el directorio `/var/www/html` de los contenedores.

Así pues, ejecutaremos el nuevo contenedor volviendo a ejecutar `compose`. Cuidado pues se debe ejecutar el comando en el mismo directorio donde tengamos nuestro archivo `docker-compose.yml`:

```
cd /home/usuario/practica6-2
```

```
docker-compose up -d
```

Y comprobamos que los contenedores están corriendo:

```
docker ps
```


Debiendo ver algo como:

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
82c8baf15221	docker-project_nginx	"/docker-entrypoint...."	23 seconds ago	Up 22 seconds	0.0.0.0:80->80/tcp, :::80->80/tcp
10778c6686d8	php:7.0-fpm	"docker-php-entrypoi..."	25 seconds ago	Up 23 seconds	9000/tcp

Y si ahora volvemos a acceder a `http://IP_Maq_Virtual`, veremos la página `Hola mundo`:

4. Creación de un contenedor para datos

Como véis, hemos montado el directorio `www/html` en ambos contenedores, el de `nginx` y el de `php`. Sin embargo, esta no es una forma adecuada de hacerlo. En este paso crearemos un contenedor independiente que se encargará de contener los datos y lo enlazaremos con el resto de contenedores.

Para llevar a cabo esta tarea, volvemos a editar el `docker-compose.yml`:

```
nano /usuario/home/practica6-2/docker-compose.yml
```

Y añadiremos un nuevo servicio a los que ya teníamos, quedando así:

```
nginx:
  build: ./nginx/
  container_name: nginx-container
  ports:
    - 80:80
  links:
    - php
  volumes_from:
    - app-data

php:
  image: php:7.0-fpm
  container_name: php-container
  expose:
    - 9000
  volumes_from:
    - app-data

app-data:
  image: php:7.0-fpm
  container_name: app-data-container
  volumes:
    - ./www/html:/var/www/html/
  command: "true"
```

Así que para recrear y lanzar todos los contenedores ejecutamos de nuevo (recordad, dentro del directorio donde se encuentra el archivo):

```
docker-compose up -d
```

Y volvemos a verificar que están corriendo todos:

```
docker ps -a
```

Debiendo ver algo como:

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
849315c7ffc0	docker-project_nginx	"/docker-entrypoint..."	27 seconds ago	Up 25 seconds	0.0.0.0:80->80/tcp, :::80->80/tcp
59a0d7040fd8	php:7.0-fpm	"docker-php-entrypoi..."	28 seconds ago	Up 27 seconds	9000/tcp
fbca95944234	php:7.0-fpm	"docker-php-entrypoi..."	29 seconds ago	Exited (0) 28 seconds ago	

5. Creación de un contenedor MySQL

En esta sección crearemos un contenedor de una base de datos MySQL y lo enlazaremos con el resto de contenedores.

Primero, modificaremos la imagen PHP e instalaremos la extensión PHP para MySQL, de tal forma que nos permita conectarnos desde nuestra aplicación PHP a nuestra BBDD MySQL.

Creemos, si no lo teníamos ya, nuestro directorio *php* y dentro de él, el archivo Dockerfile:

```
mkdir /home/usuario/practica6-2/php
nano /home/usuario/practica6-2/php/Dockerfile
```

Y dentro del Dockerfile ponemos:

```
FROM php:7.0-fpm
RUN docker-php-ext-install pdo_mysql
```

Y una vez más, debemos editar `docker-compose.yml` con el objetivo de que se creen el contenedor para MySQL y el contenedor de los datos de MySQL que contendrá la base de datos y las tablas:

```
services:
  nginx:
    build: ./nginx/
    container_name: nginx-container
    ports:
      - 80:80
    links:
      - php
    volumes_from:
      - app-data
  php:
    build: ./php/
    container_name: php-container
    expose:
      - 9000
    links:
      - mysql
    volumes_from:
      - app-data
```

```

app-data:
  image: php:7.0-fpm
  container_name: app-data-container
  volumes:
    - ./www/html:/var/www/html/
  command: "true"

```

```

mysql:
  image: mysql:5.7
  container_name: mysql-container
  volumes_from:
    - mysql-data
  environment:
    MYSQL_ROOT_PASSWORD: secret
    MYSQL_DATABASE: mydb
    MYSQL_USER: myuser
    MYSQL_PASSWORD: password

```

```

mysql-data:
  image: mysql:5.7
  container_name: mysql-data-container
  volumes:
    - /var/lib/mysql
  command: "true"

```

Después de guardar este archivo, editamos el archivo `index.php` y hacemos algunos cambios para comprobar la conexión a la base de datos.

El archivo `index.php` debe quedar así:

```

<!DOCTYPE html>
<head>
  <title>¡Hola mundo!</title>
</head>

<body>
  <h1>¡Hola mundo!</h1>
  <p><?php echo 'Estamos corriendo PHP, version: ' . phpversion(); ?></p>
  <?
    $database = "mydb";
    $user = "myuser";
    $password = "password";
    $host = "mysql";

    $connection = new PDO("mysql:host={$host};dbname={$database};charset=utf8", $user, $password);
    $query = $connection->query("SELECT TABLE_NAME FROM information_schema.TABLES WHERE
TABLE_TYPE='BASE TABLE'");
    $tables = $query->fetchAll(PDO::FETCH_COLUMN);

    if (empty($tables)) {
      echo "<p>No hay tablas en la base de datos \"{$database}\".</p>";
    } else {
      echo "<p>La base de datos \"{$database}\" tiene las siguientes tablas:</p>";
      echo "<ul>";
      foreach ($tables as $table) {
        echo "<li>{$table}</li>";
      }
      echo "</ul>";
    }
  ?>
</body>
</html>

```

Guardad el archivo y lanzad los contenedores una vez más:

```
docker-compose up -d
```

Y verificamos que están ejecutándose:

```
docker ps -a
```

Y veremos:

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
d3e82747fe0d	mysql:5.7	"docker-entrypoint.s..."	39 seconds ago	Up 38 seconds	3306/tcp, 33060/tcp
mysql-container					
606320e5a7f8	mysql:5.7	"docker-entrypoint.s..."	41 seconds ago	Exited (0) 39 seconds ago	
mysql-data-container					
ca4f63797d11	docker-project_php	"docker-php-entrypoi..."	2 hours ago	Up 2 hours	9000/tcp
php-container					
849315c7ffc0	docker-project_nginx	"/docker-entrypoint..."	2 hours ago	Up 2 hours	0.0.0.0:80->80/tcp,
:::80->80/tcp	nginx-container				
fbca95944234	php:7.0-fpm	"docker-php-entrypoi..."	2 hours ago	Exited (0) 39 seconds ago	
app-data-					

6. Verificación de conexión a la base de datos

Si ahora accedemos a http://IP_Maq_Virtual, deberíamos obtener la siguiente pantalla:

¡Hola mundo!

Estamos corriendo PHP, version: 7.0.33

No hay tablas en la base de datos "mydb".

Como podéis ver, nos dice que no tenemos ninguna tabla en la base de datos *mydb*.

Sin embargo, el hecho es que realmente sí existen algunas tablas, simplemente no son visibles para un usuario normal. Si quisiéramos verlas, debemos editar el archivo `index.php` y cambiar `$user` por `root` y `$password` a `secret`.

Es decir:

```
nano /home/usuario/www/html/index.php
```

Y cambiar las líneas:

```
$user = "root";  
$password = "secret";
```

Guardad el archivo y refrescad la página. Deberías obtener ahora una pantalla con todas las tablas de la base de datos, tal que así:

¡Hola mundo!

Estamos corriendo PHP, version: 7.0.33

La base de datos "mydb" tiene las siguientes tablas:

- columns_priv
- db
- engine_cost
- event
- func
- general_log
- gtid_executed
- help_category
- help_keyword
- help_relation
- help_topic
- innodb_index_stats
- innodb_table_stats
- ndb_binlog_index
- plugin
- proc
- procs_priv
- proxies_priv
- server_cost
- servers
- slave_master_info
- slave_relay_log_info
- slave_worker_info
- slow_log
- tables_priv
- time_zone
- time_zone_leap_second
- time_zone_name
- time_zone_transition
- time_zone_transition_type
- user
- accounts
- cond_instances
- events_stages_current
- events_stages_history
- events_stages_history_long
- events_stages_summary_by_account_by_event_name

Tarea

Documenta, incluyendo capturas de pantallas, el proceso que has seguido para realizar el despliegue de esta nueva aplicación, así como el resultado final.

Práctica 3: Despliegue de servidores web con usuarios autenticados mediante LDAP usando Docker y docker-compose

Introducción

¿Qué es un servicio de directorio LDAP?

LDAP (Lightweight Directory Access Protocol) o también conocido como «Protocolo Ligero de Acceso a Directorios» es un protocolo de la capa de aplicación TCP/IP que permite el acceso a un servicio de directorio ordenado y distribuido, para buscar cualquier información en un entorno de red.

Aclaración

Un directorio es un conjunto de objetos con atributos que están organizados de manera lógica y jerárquica, es decir, está en forma de árbol y perfectamente ordenado en función de lo que nosotros queramos, ya sea alfabéticamente, por usuarios, direcciones etc.

Generalmente un servidor LDAP se encarga de almacenar información de autenticación, es decir, el usuario y la contraseña, para posteriormente dar acceso a otro protocolo o servicio del sistema. Además de almacenar el nombre de usuario y la contraseña, también puede almacenar otra información como datos de contacto del usuario, ubicación de los recursos de la red local, certificados digitales de los propios usuarios y mucho más.

LDAP es un protocolo que nos permite acceder a los recursos de la red local, sin necesidad de crear los diferentes usuarios en el sistema operativo, además, es mucho más versátil. Por ejemplo, LDAP permite realizar tareas de autenticación y autorización a usuarios de diferentes softwares como Docker, OpenVPN, servidores de archivos como los usados por QNAP, Synology o ASUSTOR entre otros, y muchos más usos.

LDAP puede ser utilizado tanto por un usuario al que se pide unos credenciales de acceso, como también por las aplicaciones para saber si tienen acceso a determinada información del sistema o no. Generalmente un servidor LDAP se encuentra en una red privada, es decir, redes de área local, para autenticar las diferentes aplicaciones y usuarios, pero también podría funcionar sobre redes públicas sin ningún problema.

Info

En definitiva, LDAP nos proporciona un servicio de autenticación y autorización para poder acceder a distintos recursos en red, como por ejemplo, a un sitio web.

Tenemos, por tanto, la posibilidad de utilizar otra autenticación centralizada para el mismo cometido con LDAP.

Implementaciones de LDAP

Microsoft Active Directory utiliza internamente el protocolo LDAP para realizar todas las comunicaciones desde los clientes hasta el servidor o servidores, por lo tanto, se encarga de que los clientes puedan autenticarse y acceder a cualquier dato almacenado, además, debemos tener en cuenta que este protocolo es multiplataforma, no solamente lo tenemos en sistemas operativos Windows sino que también es compatible con Linux, Unix y macOS, todo ello a través del protocolo. Para que os hagáis una idea, los siguientes servicios de directorio usan este protocolo para su comunicación:

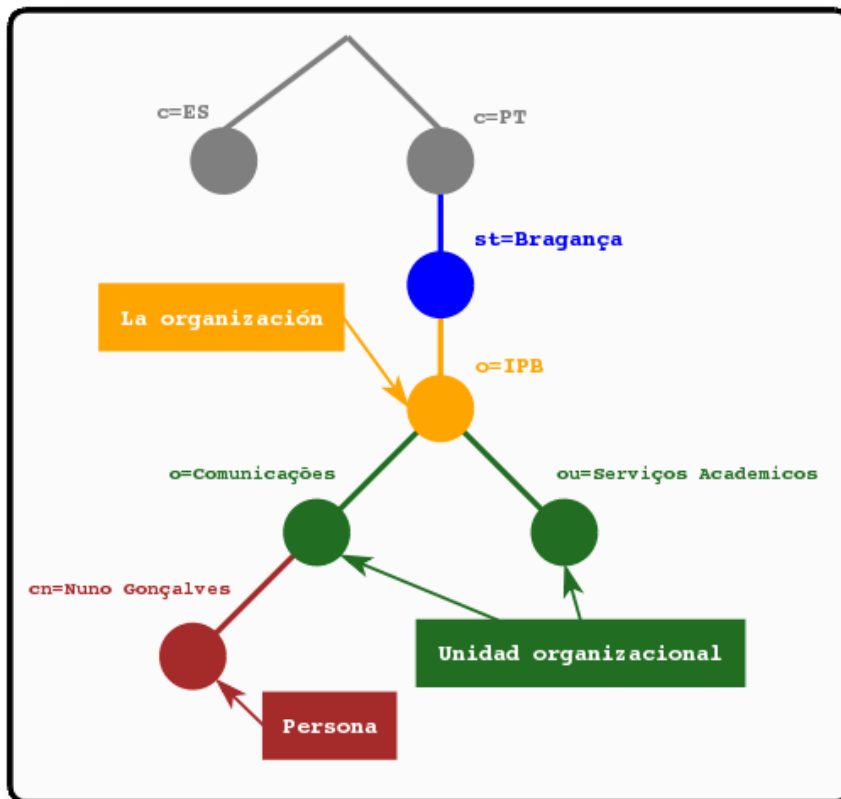
- Active Directory de Microsoft
- Apache
- Servicio de directorio de Red Hat
- OpenLDAP

Y muchos otros servicios también lo usan, sobre todo el último, OpenLDAP, el cual es una implementación de código abierto del protocolo y que se puede instalar en cualquier sistema, ya que está disponible el código fuente para compilarlo. No obstante, en la mayoría de distribuciones de Linux lo tenemos disponible en sus repositorios.

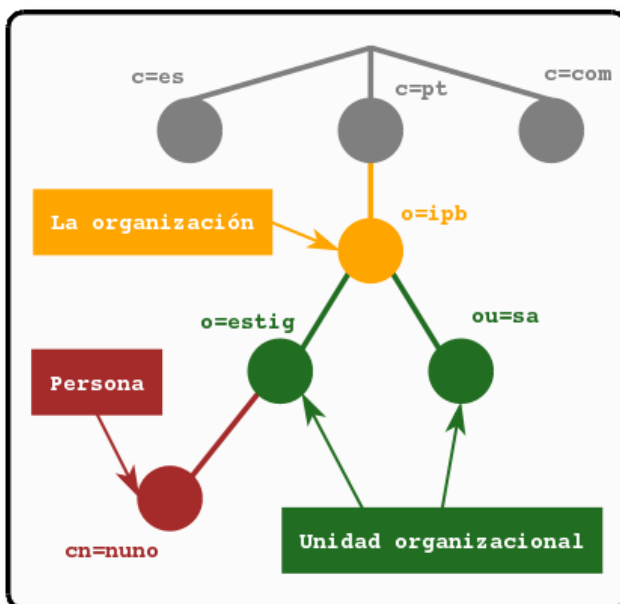
¿Cómo se organiza la información en LDAP?

En LDAP, las entradas están organizadas en una estructura jerárquica en árbol. Tradicionalmente, esta estructura reflejaba los límites geográficos y organizacionales.

Las entradas que representan países aparecen en la parte superior del árbol. Debajo de ellos, están las entradas que representan los estados y las organizaciones nacionales. Debajo de éstas, pueden estar las entradas que representan las unidades organizacionales, empleados, impresoras, documentos o todo aquello que pueda imaginarse. La siguiente figura muestra un ejemplo de un árbol de directorio LDAP haciendo uso del nombramiento tradicional.



El árbol también se puede organizar basándose en los nombres de dominio de Internet. Este tipo de nombramiento es muy popular, ya que permite localizar un servicio de directorio haciendo uso de los DNS. La siguiente figura muestra un ejemplo de un directorio LDAP que hace uso de los nombres basados en dominios.



¿Cómo se referencia la información?

Una entrada es referenciada por su nombre distinguido (DN), que es construido por el nombre de la propia entrada (llamado Nombre Relativo Distinguido o RDN) y la concatenación de los nombres de las entradas que le anteceden.

Por ejemplo, la entrada para Nuno Gonçalves en el ejemplo del nombramiento de Internet anterior tiene el siguiente RDN: uid=nuno y su DN sería: uid=nuno,ou=estig,dc=ipb,dc=pt.

Cómo se accede a la información?

LDAP define operaciones para interrogar y actualizar el directorio. Provee operaciones para añadir y borrar entradas del directorio, modificar una entrada existente y cambiar el nombre de una entrada. La mayor parte del tiempo, sin embargo, LDAP se utiliza para buscar información almacenada en el directorio. Las operaciones de búsqueda de LDAP permiten buscar entradas que concuerdan con algún criterio especificado por un filtro de búsqueda. La información puede ser solicitada desde cada entrada que concuerda con dicho criterio.

Por ejemplo, imaginemos que queremos buscar en el subárbol del directorio que está por debajo de dc=ipb,dc=pt a personas con el nombre Nuno Gonçalves, obteniendo la dirección de correo electrónico de cada entrada que concuerde. LDAP permite hacer esto fácilmente. O tal vez preferimos buscar las organizaciones que posean la cadena IPB en su nombre, posean número de fax y estén debajo de la entrada st=Bragança,c=PT. LDAP permite hacer esto también.

LDAP ofrece una autenticación y autorización optimizadas y una búsqueda eficaz de datos de direcciones y de usuarios. Debido a sus muchas ventajas para las empresas. LDAP sirve a modo de un estándar de la industria y es compatible con la mayoría de los productos de software. Las ventajas principales son la rapidez de las consultas y conexiones, un lenguaje de consulta sencillo y un protocolo claramente estructurado

Módulos en Apache

Un módulo es una parte independiente de un programa. La mayor parte de la funcionalidad de Apache está contenida en módulos que pueden incluirse o excluirse. Como decimos, existen una gran cantidad de Módulos para utilizarse con Apache, algunos ejemplo son: "Virtual Hosting", "Mod_JK(Java)" y "Rewrite".

Una de las principales razones de emplear módulos en Apache, es que no toda instalación requiere de las mismas funcionalidades, esto es, una instalación que utilice PHP probablemente no requiera de Tomcat (Java), o bien posiblemente no todas las instalaciones requieran de "Virtual Hosting".

Así las cosas, para no incluir todas las funcionalidades de Apache, necesarias e innecesarias para cada ocasión, en un único paquete de instalación que lo haría demasiado grande en tamaño y pesado en recursos, se hace uso de los módulos, de tal forma que sólo cargaremos en memoria los que nos hagan falta en cada ocasión.

Los módulos le permiten a los administradores del servidor activar y desactivar funcionalidades adicionales. Apache tiene módulos de seguridad, almacenamiento en caché, reescritura de URL, autenticación de contraseña y más.

Info

En Apache hay dos tipos de módulos:

- **Estáticos:** Son añadidos al compilar el servidor.
- **Dinámicos:** Se cargan dinámicamente al iniciar el servidor.

Se puede habilitar cualquiera de los módulos de la lista con el comando `a2enmod` (nombre del módulo) (usando el `sudo` si no se es superusuario), y deshabilitar cualquiera de ellos previamente habilitado mediante el comando `a2dismod` (nombre del módulo) (usando el `sudo` si no se es superusuario).

Módulos en Nginx

Los módulos estáticos existen desde sus inicios en Nginx y los dinámicos desde la versión 1.9.11 (Febrero de 2016).

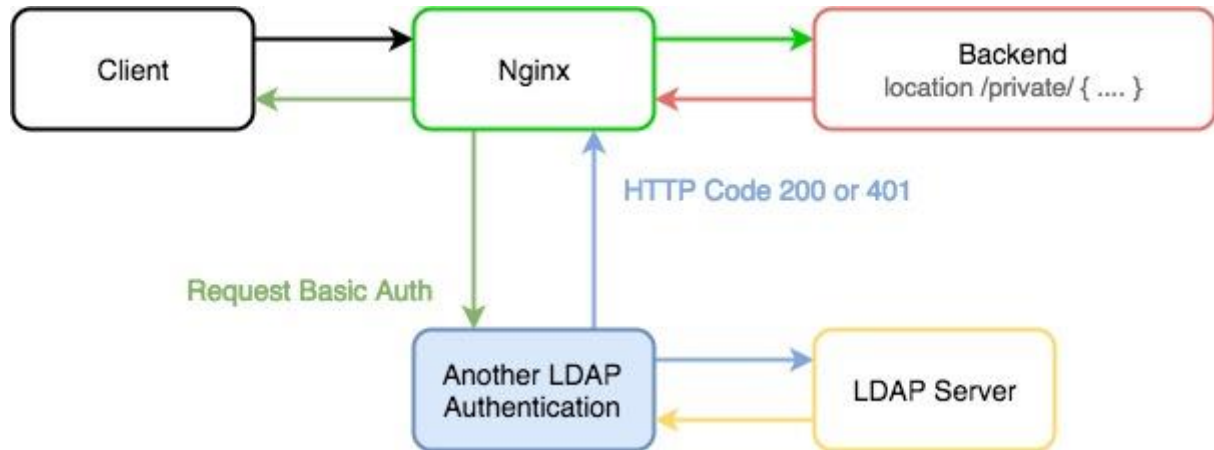
Nginx es, de hecho, una colección de módulos. Incluso funciones básicas tales como HTTP o servir ficheros estáticos dentro de HTTP, están implementadas por módulos.

Se puede extender la funcionalidad de Nginx añadiendo módulos propios. Esta arquitectura modular permite modificar fácilmente Nginx.

Módulo autenticación LDAP en Nginx

La solución aprovecha el módulo `ngx_http_auth_request_module` de Nginx y NGINX, que reenvía las peticiones de autenticación a un servicio externo. En la implementación de referencia, ese servicio es un demonio que llamamos `ldap-auth`. Está escrito en Python y se comunica con un servidor de autenticación del Protocolo Ligero de Acceso a Directorios (LDAP) - OpenLDAP por defecto, pero hemos probado el demonio `ldap-auth` también con configuraciones por defecto de Microsoft® Windows® Server Active Directory (tanto la versión 2003 como la 2012).

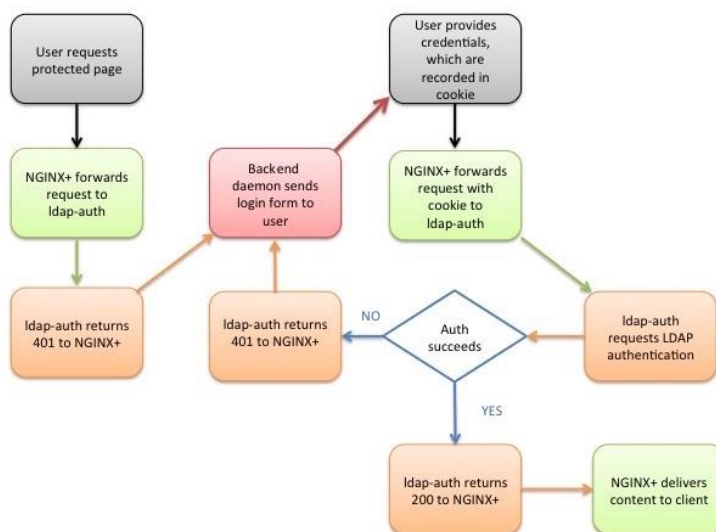
Para realizar la autenticación, el módulo `http_auth_request` realiza una subconsulta HTTP al demonio `ldap-auth`, que actúa como intermediario e interpreta la subconsulta para el servidor LDAP - utiliza HTTP para la comunicación con Nginx y la API apropiada para la comunicación con el servidor LDAP.



Para realizar la autenticación, el módulo `http_auth_request` realiza una subconsulta HTTP al demonio `ldap-auth`, que actúa como intermediario e interpreta la subconsulta para el servidor LDAP - utiliza HTTP para la comunicación con Nginx y la API apropiada para la comunicación con el servidor LDAP.

A continuación se describe paso a paso el proceso de autenticación en la implementación de referencia. Los detalles se determinan por los ajustes en el archivo de configuración `nginx-ldap-auth.conf`; ver Configuración de la implementación de referencia más abajo.

El diagrama de flujo debajo de los pasos resume el proceso.



1. Un cliente envía una solicitud HTTP para un recurso protegido alojado en un servidor para el que Nginx está actuando como proxy inverso.
2. Nginx (concretamente, el módulo *http_auth_request*) reenvía la solicitud al demonio *ldap-auth*, que responde con el código HTTP 401 porque no se han proporcionado credenciales.
3. Nginx reenvía la solicitud a *http://backend/login*, que corresponde al demonio del backend. Escribe el URI de la solicitud original en la cabecera X-Target de la solicitud reenviada.
4. El demonio del backend envía al cliente un formulario de inicio de sesión (el formulario está definido en el código Python del demonio). Tal y como se configura en la directiva *error_page*, NGINX establece el código HTTP del formulario de inicio de sesión en 200.
5. El usuario rellena los campos *Nombre de usuario* y *Contraseña* en el formulario y hace clic en el botón Login. Según el código del formulario, el cliente genera una petición HTTP POST dirigida a */login*, que Nginx reenvía al demonio del backend.
6. El demonio del backend construye una cadena con el formato *nombre de usuario:contraseña*, aplica la codificación Base64, genera una cookie llamada **nginxauth** con su valor establecido a la cadena codificada, y envía la cookie al cliente. Establece el flag *httponly* para evitar el uso de JavaScript para leer o manipular la cookie (protegiendo contra la vulnerabilidad cross-site scripting [XSS]).
7. El cliente retransmite su solicitud original (del paso 1), esta vez incluyendo la cookie en el campo Cookie de la cabecera HTTP. Nginx reenvía la solicitud al demonio *ldap-auth* (como en el paso 2).
8. El demonio *ldap-auth* decodifica la cookie y envía el nombre de usuario y la contraseña al servidor LDAP en una solicitud de autenticación.
9. La siguiente acción depende de si el servidor LDAP autentifica con éxito al usuario:
 - Si la autenticación tiene éxito, el demonio *ldap-auth* envía el código HTTP 200 a Nginx. Nginx solicita el recurso al demonio del backend y éste devuelve el código del sitio web.

El archivo *nginx-ldap-auth.conf* incluye directivas para el almacenamiento en caché de los resultados del intento de autenticación.
 - Si la autenticación falla, el demonio *ldap-auth* envía el código HTTP 401 a Nginx. Nginx reenvía la solicitud al demonio del backend de nuevo (como en el paso 3), y el proceso se repite.

Despliegue con Docker de NGINX + demonio de autenticación LDAP + OpenLDAP

Para esta práctica nos crearemos un directorio que contendrá nuestro index.html, con un texto muy simple:

```
$ mkdir app

$ cat << EOF > app/index.html
<html>
<body>
<h1>¡Hola Mundo!</h1>
</body>
</html>
EOF
```

Así como otro directorio, con el contenido de la configuración pertinente de Nginx:

```
$ mkdir conf

$ cat << EOF > conf/ldap_nginx.conf
server {
    listen 8080;

    location = / {
        auth_request /auth-proxy;
    }

    location = /auth-proxy {
        internal;

        proxy_pass http://nginx-ldap:8888;

        # URL y puerto para conectarse al servidor LDAP
        proxy_set_header X-Ldap-URL "ldap://openldap:1389";

        # Base DN
        proxy_set_header X-Ldap-BaseDN "dc=example,dc=org";

        # Bind DN
        proxy_set_header X-Ldap-BindDN "cn=admin,dc=example,dc=org";

        # Bind password
        proxy_set_header X-Ldap-BindPass "adminpassword";
    }
}
```

En esta configuración le decimos a Nginx:

- Que escuche en el puerto 8080 las peticiones HTTP
- Que cuando se acceda al sitio web, se solicite autorización en el directorio del sitio web llamado */auth-proxy*
- Se crea un nuevo location para ese directorio */auth-proxy* y que es donde se realizará la configuración de cómo conectarnos a nuestro openldap, de acuerdo con la [documentación oficial de Nginx](#) a propósito de su módulo de autenticación:
- Se indica la URL de nuestro openldap (es el nombre del contenedor que hemos levantado, ya que Docker tiene un DNS propio entre sus contenedores)

- El DN (Nombre distinguido) base sobre el que se realizarán las búsquedas en openldap
- El usuario y contraseña con el que nos conectaremos al openldap para realizar las consultas

Y ahora, procedemos con el siguiente `docker-compose.yml`:

```
version: '2'

services:
  nginx-ldap: #
    image: bitnami/nginx-ldap-auth-daemon #
    ports: #
      - 8888:8888
  nginx: #
    image: bitnami/nginx
    ports:
      - 8080:8080
    volumes: #
      - ./app:/app
      - ./conf/ldap_nginx.conf:/opt/bitnami/nginx/conf/server_blocks/ldap_nginx.conf
  openldap: #
    image: bitnami/openldap
    ports:
      - '1389:1389'
    environment: #
      - LDAP_ADMIN_USERNAME=admin
      - LDAP_ADMIN_PASSWORD=adminpassword
      - LDAP_USERS=customuser
      - LDAP_PASSWORDS=custompassword
```

Tras esto sólo queda ejecutar `compose`:

```
docker-compose up
```

Y comprobar que no se producen errores.

Despliegue con Docker de PHP + Apache con autenticación LDAP

1. Creamos un directorio que se llame `Practica6.3`
2. En primer lugar, como es obvio, dentro del directorio creado debemos crear el `index.php` de nuestra aplicación:

```
<?php
echo "Well, hello LDAP authenticated user!";
```

3. Dentro de nuestro directorio de trabajo, creado anteriormente, crearemos otro directorio llamado `Docker` y dentro de él, un `Dockerfile` (`./Docker/Dockerfile`)

Nota

En Linux, cuando queremos hacer referencia al directorio actual, lo hacemos con un punto `.`

Si dentro de nuestro directorio actual tenemos una carpeta llamada `prueba`, podemos hacer referencia a ella como `./prueba`, ya que el `.` hace referencia precisamente al directorio donde nos encontramos.

Para completar este `Dockerfile` con las opciones/directivas adecuadas, leed los comentarios y podéis apoyaros en la teoría o en [este cheatsheet](#):

```
# ./Docker/Dockerfile --> directorio donde se encuentra este archivo

# Imagen base sobre la que vamos a trabajar
____ php:7-apache

# Activamos el módulo LDAP de Apache ejecutand el siguiente comando
____ a2enmod authnz_ldap

# Añadimos las reglas/configuración de LDAP al directorio conf-enabled de Apache
# (crearemos este archivo en el siguiente paso)
____ Docker/ldap-demo.conf /etc/apache2/conf-enabled/

# Añadimos ayuda de depuración (debugging) en la configuración de apache
# En caso de necesitarlo, lo descomentamos para ejecutar el siguiente comando
# ____ echo "LogLevel debug" >> apache2.conf

# Establecemos el directorio de trabajo adecuado
____ /var/www/html/demo

# Configuramos Apache para usar la configuración ldap definida arriba, la copiamos de nuestro ordenador al
contenedor
____ Docker/.htaccess ./htaccess

# Copiamos los archivos del proyecto que necesitamos, al contenedor
____ index.php ./
```

Tarea

Completa el `Dockerfile`.

4. Ahora crearemos el archivo `./Docker/ldap-demo.conf`, que es la configuración LDAP. Aquí establecemos los criterios de conexión con el contenedor de Openldap, password y URL.

Las directivas `PassEnv` al principio del archivo nos permiten omitir nuestras credenciales y pasarlas luego como variables de entorno al correr la imagen del contenedor:

```
# ./Docker/ldap-demo.conf
PassEnv LDAP_BIND_ON
PassEnv LDAP_PASSWORD
PassEnv LDAP_URL
<AuthnProviderAlias ldap demo>
  AuthLDAPBindDN ${LDAP_BIND_ON}
  AuthLDAPBindPassword ${LDAP_PASSWORD}
  AuthLDAPURL ${LDAP_URL}
</AuthnProviderAlias>
```

5. Creamos el archivo `.htaccess`:

```
# ./htaccess
AuthBasicProvider demo
AuthType Basic
AuthName "Protected Area"
Require valid-user
```

6. Dentro de nuestro directorio de trabajo, construimos la imagen con el siguiente comando:

```
docker build \
-t docker-ldap \
-f ./Docker/dockerfile \
.
```

7. Corremos el contenedor indicando las credenciales de nuestra cuenta LDAP mediante variables de entorno con la flag `-e`. Para este caso, vamos a probar un servidor LDAP externo, simulando que tuviéramos que integrar nuestro despliegue con un servidor ya existente en la empresa. Utilizaremos un servidor público en Internet dedicado a pruebas: <https://www.forumsys.com/2022/05/10/online-ldap-test-server/>

```
docker run \
-p 3000:80 \
--name ldap_demo \
-e LDAP_BIND_ON='cn=read-only-admin,dc=example,dc=com' \
-e LDAP_PASSWORD='password' \
-e LDAP_URL='LDAP://ldap.forumsys.com/dc=example,dc=com' \
docker-ldap
```

8. No nos queda más que visitar `http://IP-Máq-Debian:3000/demo`. Si todo ha ido bien, nos solicitará nuestras credenciales para loguearnos contra el servidor openldap.

Tarea

Documenta todo el proceso de la práctica, siguiendo los pasos y con las capturas y explicaciones pertinentes.

Referencias

[Para qué sirve el protocolo LDAP y cómo funciona](#)

[OpenLDAP conceptos teóricos](#)

[Using Nginx and NGINX to Authenticate Application Users with LDAP](#)

[Simple Docker/Apache/PHP Authentication with LDAP](#)

[bitnami/nginx-ldap-auth-daemon](#)

[nginxinc/nginx-ldap-auth](#)

[Dockerfile cheatsheet](#)

[Difference between RUN and CMD in a Dockerfile](#)